

Versi 2.0
September 2026



PEMROGRAMAN WEB

MODUL 5 - AUTENTIKASI, MIDDLEWARE & FILE STORAGE

Disusun oleh:

Christian Sri Kusuma Aditya, S.Kom., M.Kom

Shofwan Alif Alfani

Nisrina Nurhafizhah

LABORATORIUM INFORMATIKA
Universitas Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI.....	2
PENDAHULUAN.....	3
TUJUAN.....	3
TARGET MODUL.....	3
PERSIAPAN.....	3
KATA KUNCI.....	3
MATERI.....	4
1. Autentikasi.....	4
1.1. Autentikasi dan otorisasi pada aplikasi web.....	4
1.2. Session dan token-based authentication.....	5
1.3. JSON Web Token (JWT).....	6
2. Implementasi autentikasi JWT menggunakan Laravel.....	7
2.1. Instalasi dan konfigurasi JWT.....	7
2.2. Membuat endpoint autentikasi.....	11
3. Middleware dan proteksi API.....	15
3.1. Konsep middleware.....	15
3.2. Public dan protected route.....	16
3.3. Bearer token dan pengujian API.....	17
4. File storage dan file upload.....	25
4.1. Konsep file storage.....	25
4.2. Public storage dan akses file.....	25
4.3. File upload.....	26
5. CORS dan pengenalan integrasi API ke frontend.....	30
5.1. Cross-Origin Resource Sharing (CORS).....	30
5.2. Pengenalan Integrasi API dengan client.....	32
5.3. Sekilas frontend framework.....	36
CODELAB.....	37
DEMO.....	38
RUBRIK PENILAIAN.....	39



PENDAHULUAN

TUJUAN

1. Memahami konsep autentikasi pada aplikasi web dan REST API
2. Memahami konsep JSON Web Token (JWT) sebagai mekanisme autentikasi
3. Memahami penggunaan middleware untuk mengatur dan membatasi akses ke endpoint API
4. Memahami konsep file storage dan file upload di Laravel
5. Memahami dasar integrasi REST API Laravel dengan client atau frontend

TARGET MODUL

1. Praktikan mampu mengimplementasikan autentikasi berbasis JWT
2. Praktikan mampu menerapkan middleware untuk membedakan public route dan protected route
3. Praktikan mampu melakukan upload, validasi, dan penyimpanan file pada Laravel
4. Praktikan mampu memahami cara komunikasi antara Laravel REST API (backend) dengan frontend

PERSIAPAN

1. Bagi mahasiswa Muslim, disunnahkan membaca doa berikut sebelum memulai praktikum
رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Laptop pribadi atau komputer
3. PHP & Composer
4. Laravel framework
5. Motivasi dan komitmen belajar yang kuat

KATA KUNCI

Autentikasi, JWT, Middleware, REST API, Bearer Token, File Storage, File Upload, CORS, Laravel

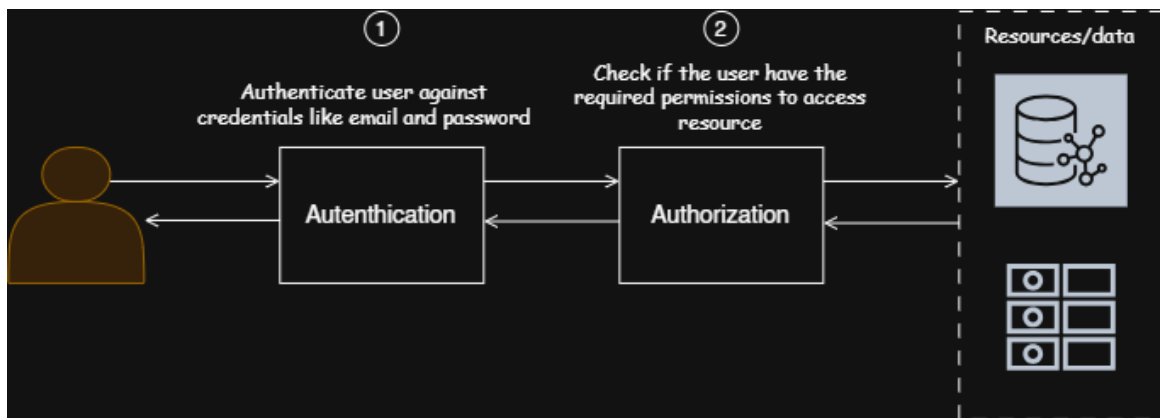


MATERI

1. Autentikasi

1.1. Autentikasi dan otorisasi pada aplikasi web

Pada aplikasi web, tidak semua pengguna memiliki hak akses terhadap seluruh data dan fitur yang tersedia. Beberapa informasi dapat diakses oleh siapa saja, sedangkan informasi tertentu hanya boleh diakses oleh pengguna yang sudah login. Inilah yang disebut autentikasi dan otorisasi. Autentikasi adalah proses yang dilakukan untuk memverifikasi identitas pengguna. Melalui autentikasi, aplikasi dapat mengetahui apakah pengguna yang melakukan request ke server merupakan pengguna yang telah terdaftar dan memiliki kredensial yang valid. Setelah identitas pengguna berhasil diverifikasi, aplikasi perlu menentukan apa saja yang dapat dilakukan oleh pengguna tersebut. Di sinilah konsep otorisasi digunakan. Otorisasi adalah proses untuk menentukan apa saja yang boleh dilakukan oleh pengguna tersebut. Jika digambarkan secara sederhana, prosesnya kurang lebih seperti ini:



Gambar 1.1: Alur proses autentikasi

Jika digambarkan dengan pertanyaan sederhana:

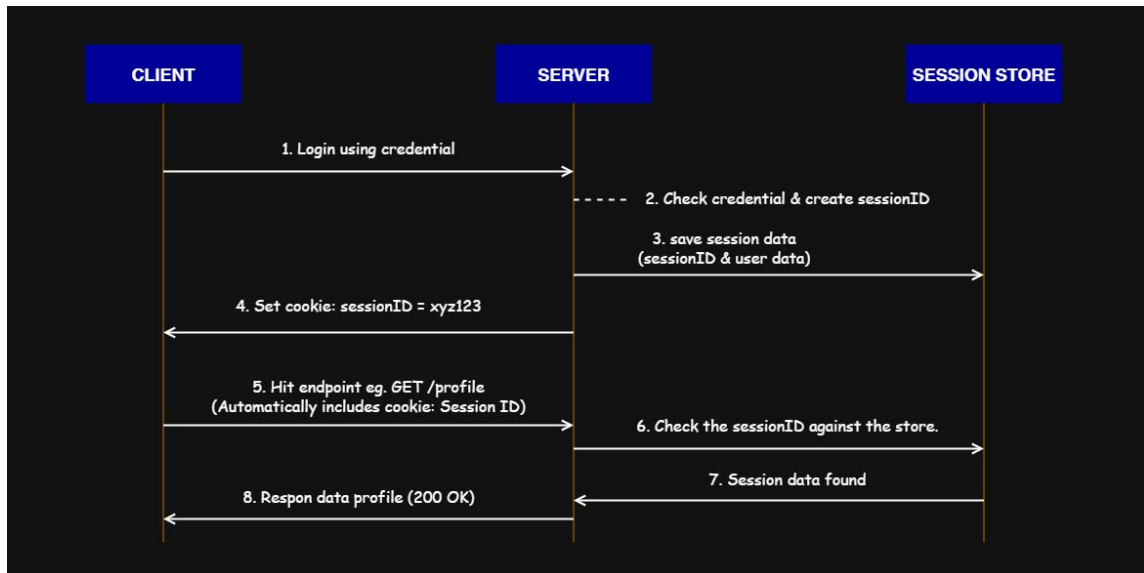
Tabel 1: Gambaran autentikasi dan otorisasi

Konsep	Pertanyaan	Contoh
Autentikasi	Siapa pengguna ini?	Login dengan email dan password
Otorisasi	Apa yang boleh dilakukan?	Boleh menghapus data tertentu



1.2. Session dan token-based authentication.

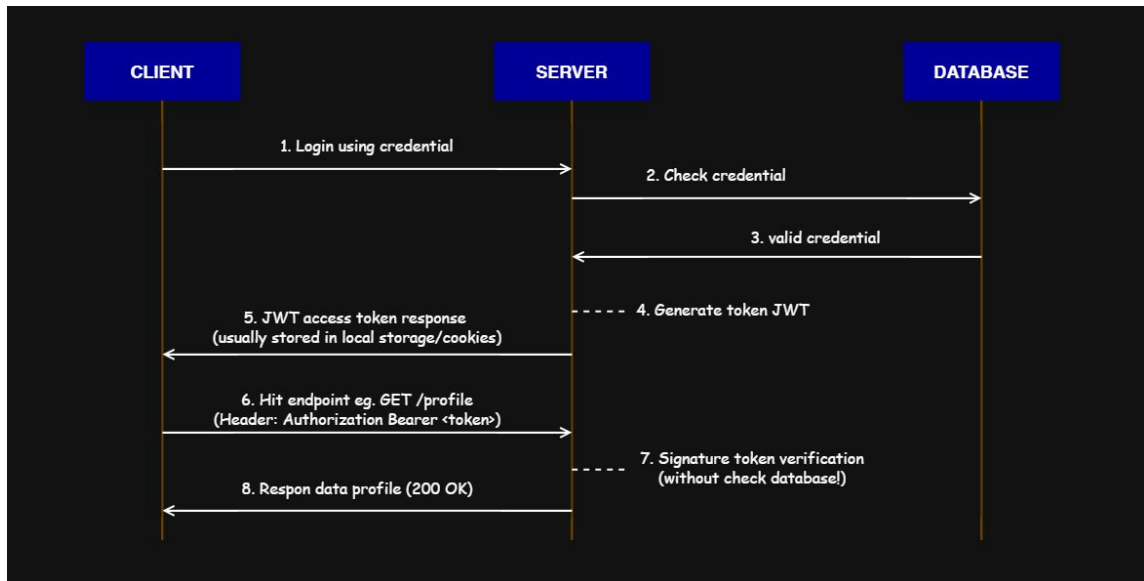
Salah satu pendekatan yang umum digunakan untuk autentikasi pada aplikasi web adalah *session-based authentication*. Pada pendekatan ini, server membuat dan menyimpan data session pengguna di memori atau database setelah proses login berhasil. Selanjutnya, browser akan menyimpan dan mengirimkan identifikasi sesi tersebut (Session ID) pada setiap request berikutnya, umumnya melalui cookie. Dengan session ID ini, server dapat mengenali identitas pengguna yang melakukan request. Jika digambarkan secara sederhana, prosesnya seperti ini:



Gambar 1.2: Alur session-based authentication

Selain session, terdapat pendekatan lain, yaitu token-based authentication. Pada pendekatan ini, server memberikan token kepada client setelah proses autentikasi berhasil. Token tersebut kemudian disimpan oleh client dan dilampirkan kembali pada setiap request berikutnya melalui HTTP Header Authorization. Server kemudian memvalidasi token tersebut untuk memastikan keabsahan identitas pengguna tanpa perlu memeriksa data session di database. Jika digambarkan secara sederhana, prosesnya seperti ini:





Gambar 1.3: Alur token-based authentication

1.3. JSON Web Token (JWT)

Salah satu teknologi yang populer digunakan untuk token-based authentication dan akan diimplementasikan pada modul kali ini adalah JSON Web Token (JWT). JWT merupakan format token yang dapat membawa informasi tertentu antara client dan server. Secara umum, struktur JWT terdiri dari tiga bagian, yaitu Header, Payload, dan Signature yang masing-masing akan dijelaskan berikut:

Header

Header berisi informasi mengenai token, salah satunya adalah algoritma yang digunakan untuk menghasilkan signature. Contoh:

```

{
  "alg": "HS256",
  "typ": "JWT"
}
    
```

Gambar 1.4: Header JWT



Payload

Payload berisi klaim atau informasi yang dibawa oleh token. Informasi pada payload inilah yang akan digunakan untuk membantu server mengenali token dan pengguna yang terkait. Contoh:

```
{
  "sub": "1234567890",
  "name": "Shofwan Alif",
  "admin": true,
  "iat": 1516239022
}
```

Gambar 1.5: Payload JWT

Signature

Signature digunakan untuk membantu memastikan bahwa token tidak berubah setelah dibuat.

```
secret signature 256 bits long
```

Gambar 1.6: Signature JWT

2. Implementasi autentikasi JWT menggunakan Laravel

Masih di repository project yang sama di modul sebelumnya, ikuti step berikut:

2.1. Instalasi dan konfigurasi JWT

Jalankan command `composer require tymon/jwt-auth` dan tunggu proses instalasinya sampai selesai

```
- Installing tymon/jwt-auth (2.3.0): Extracting archive
Generating optimized autoload files
> Illuminate\Foundation\ComposerScripts::postAutoloadDump
> @php artisan package:discover --ansi

INFO Discovering packages.

laravel/pail ..... DONE
laravel/sail ..... DONE
laravel/sanctum ..... DONE
laravel/tinker ..... DONE
nesbot/carbon ..... DONE
nunomaduro/collision ..... DONE
nunomaduro/termwind ..... DONE
tymon/jwt-auth ..... DONE

83 packages you are using are looking for funding.
Use the 'composer fund' command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

INFO No publishable resources for tag [laravel-assets].

No security vulnerability advisories found.
Using version '2.3' for tymon/jwt-auth
PS C:\xampp\htdocs\CODELAB-4-WEB> |
```

Gambar 2.1: Instalasi awal JWT di Laravel



Setelah berhasil, lanjutkan melakukan publish konfigurasi file dengan menjalankan command `php artisan vendor:publish`

`--provider="Tymon\JWTAuth\Providers\LaravelServiceProvider"`

```
PS C:\xampp\htdocs\CODELAB-4-WEB> php artisan vendor:publish --provider="Tymon\JWTAuth\Providers\LaravelServiceProvider"

INFO Publishing assets.
Copying file [C:\xampp\htdocs\CODELAB-4-WEB\vendor\tymon\jwt-auth\config\config.php] to [C:\xampp\htdocs\CODELAB-4-WEB\config\jwt.php] DONE
```

Gambar 2.2: Konfirmasi publish konfigurasi file berhasil

Jika perintah tersebut berhasil dijalankan, akan ada 1 file baru, yaitu config/jwt.php. Di dalam file ini kita bisa melakukan custom jika memang diperlukan, seperti mengubah waktu kedaluwarsa token, jenis algoritma, dan lain-lain.

Selanjutnya, jalankan command `php artisan jwt:secret` untuk membuat secret key JWT secara otomatis di file .env. Jika berhasil, di dalam file .env akan muncul variabel baru yaitu:

`JWT_SECRET=random_screet_key_jwt`

Lanjut untuk konfigurasi guard API, buka file auth.php di dalam folder config (config/auth.php) kemudian temukan kode berikut:

```
'guards' => [
    'web' => [
        'driver' => 'session',
        'provider' => 'users',
    ],
],
```



Tambahkan menjadi seperti ini:

```
'guards' => [
    'web' => [
        'driver' => 'session',
        'provider' => 'users',
    ],

    'api' => [
        'driver' => 'jwt',
        'provider' => 'users'
    ]
],
```

Langkah selanjutnya, kita akan melakukan sedikit modifikasi pada kode di dalam model User dengan menambahkan beberapa method. Buka file **app/model/user.php** dan ubah kodenya menjadi seperti berikut ini:

[Kode lengkap model User](#)

Modifikasi yang dilakukan meliputi:

```
use Tymon\JWTAuth\Contracts\JWTSubject;
```

Gambar 2.3: Penerapan JWTSubject pada Model User untuk mendukung autentikasi JWT



```
// Implementation user class with JWT subject
class User extends Authenticatable implements JWTSubject //<-- implements JWTSubject
{
    //...
```

Gambar 2.4: Modifikasi class User agar mendukung autentikasi JWT

```
// adding 2 methods to the User class

/**
 * Get the identifier that will be stored in the subject claim of the JWT.
 *
 * @return mixed
 */
public function getJWTIdentifier()
{
    return $this->getKey();
}

/**
 * Return a key value array, containing any custom claims to be added to the JWT.
 *
 * @return array
 */
public function getJWTCustomClaims()
{
    return [];
}
```

Gambar 2.5: Modifikasi method untuk autentikasi JWT



2.2. Membuat endpoint autentikasi

Setelah selesai dengan konfigurasi JWT, langkah selanjutnya adalah membuat endpoint untuk autentikasi secara keseluruhan, meliputi register, login, me, dan logout.

Buat controller untuk auth terlebih dahulu. Jalankan command `php artisan make:controller AuthController`. Jika berhasil, tampilannya akan seperti di bawah ini:

```
PS C:\xampp\htdocs\CODELAB-4-WEB> php artisan make:controller AuthController
INFO Controller [C:\xampp\htdocs\CODELAB-4-WEB\app\Http\Controllers\AuthController.php] created successfully.
PS C:\xampp\htdocs\CODELAB-4-WEB> |
```

Gambar 2.6: Membuat controller untuk autentikasi

File AuthController otomatis tergenerate di app/Http/Controllers.

Selanjutnya, agar konsisten kita membuat request validation terpisah untuk register terlebih dahulu dengan menjalankan command `php artisan make:request RegisterRequest`

```
PS C:\xampp\htdocs\CODELAB-4-WEB> php artisan make:request RegisterRequest
INFO Request [C:\xampp\htdocs\CODELAB-4-WEB\app\Http\Requests\RegisterRequest.php] created successfully.
PS C:\xampp\htdocs\CODELAB-4-WEB> |
```

Gambar 2.7: Membuat validasi request untuk register

File RegisterRequest otomatis tergenerate di app/Http/Requests. kode lengkap di bawah ini:

<https://pastebin.com/p4ArOEjs>

Buat request validation lagi untuk login dengan menjalankan command `php artisan make:request LoginRequest`. Jika berhasil, file LoginRequest akan otomatis tergenerate di folder yang sama dengan RegisterRequest. Kode lengkap bisa diakses pada link berikut:

<https://pastebin.com/VVKQe7A5>

Selanjutnya, buka file AuthController dan ubah kodenya dengan menambahkan beberapa method. Kode lengkap ada di link berikut:

<https://pastebin.com/Z5W3kVdD>



Penjelasan masing-masing method di class AuthController:

Method register

```
public function register(RegisterRequest $request)
{
    $user = User::create([
        'name' => $request->name,
        'email' => $request->email,
        'password' => Hash::make($request->password),
    ]);

    return response()->json([
        'message' => 'User registered successfully',
        'user' => $user
    ], 201);
}
```

Gambar 2.8: Method register

Method register digunakan untuk menerima dan mendaftarkan pengguna baru ke dalam sistem. Cara kerjanya mengalir secara berurutan. Saat ada data yang masuk, sistem akan memastikan semua input sudah sesuai aturan melalui kelas validasi RegisterRequest pada parameter. Setelah data dipastikan valid, password akan di-hash sehingga tersimpan sebagai karakter acak demi keamanan agar tidak mudah dibaca. Kemudian, seluruh data tersebut akan disimpan secara permanen dalam database. Sebagai langkah penutup, sistem akan mengirimkan respons berformat JSON yang menggambarkan status keberhasilan beserta rincian data pengguna baru yang berhasil terdaftar.



Method login

```
public function login(LoginRequest $request)
{
    $credentials = $request->only('email', 'password');

    $token = auth('api')->attempt($credentials);

    if (!$token) {

        return response()->json([
            'message' => 'Invalid email or password'
        ], 401);
    };

    return response()->json([
        'message' => 'Login successfully',
        'user' => auth('api')->user(),
        'token' => $token
    ]);
}
```

Gambar 2.9: Method login

Method login berfungsi untuk memverifikasi identitas pengguna yang ingin masuk ke dalam sistem dan memberikan token akses jika kredensialnya valid. Cara kerjanya diawali dengan menerima data email dan password yang telah dipastikan valid oleh kelas validasi LoginRequest. Sistem kemudian mengambil kedua data tersebut dan mencoba melakukan autentikasi dengan membandingkannya dengan basis data. Jika kombinasi email dan kata sandi salah, sistem akan langsung menghentikan proses dan mengembalikan respons error dengan status HTTP 401 Unauthorized. Namun, jika data cocok, sistem akan secara otomatis memberikan token akses berbasis JWT. Sebagai penutup, sistem mengirimkan respons JSON yang memberitahukan bahwa proses masuk ke sistem berhasil, lengkap dengan data detail pengguna beserta token akses yang nantinya dapat digunakan pengguna untuk mengakses fitur yang dilindungi.



Method "me"

```
public function me()
{
    return response()->json([
        'user' => auth('api')->user()
    ], 200);
}
```

Gambar 3: Method "me"

Method ini berfungsi untuk mengambil dan menampilkan profil pengguna yang sedang aktif atau sudah terautentikasi. Cara kerjanya sangat ringkas. Sistem akan memeriksa identitas pengguna berdasarkan token akses yang dikirimkan bersama permintaan tersebut. Jika token sah, sistem akan langsung mengambil informasi profil pengguna yang bersangkutan dan mengembalikannya kepada klien dalam bentuk respons JSON dengan status HTTP 200 OK.

Method logout

```
public function logout()
{
    auth('api')->logout();

    return response()->json(["message" => 'Successfully logged out']);
}
```

Gambar 3.1: Method Logout

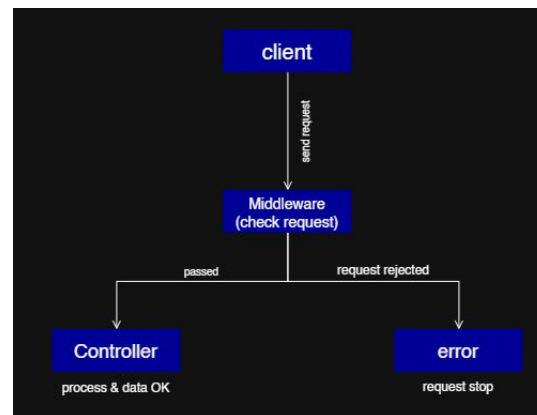
Method ini berfungsi untuk mengakhiri sesi pengguna dan membatalkan token autentikasi JWT yang sedang digunakan. Cara kerjanya cukup singkat: sistem akan membaca token JWT yang dikirim oleh pengguna pada request tersebut, lalu mematikan token tersebut melalui perintah `auth('api')->logout()`. Hal ini membuat token tersebut tidak dapat digunakan lagi untuk mengakses API. Setelah token berhasil dimatikan, sistem mengembalikan respons JSON berisi pesan konfirmasi bahwa proses keluar dari aplikasi telah berhasil.



3. Middleware dan proteksi API

3.1. Konsep middleware

Dalam arsitektur aplikasi web, sebuah request tidak selalu langsung diproses oleh controller. Sebelum mencapai controller, request dapat melewati satu atau beberapa lapisan pemeriksaan. Lapisan tersebut disebut middleware. Middleware adalah mekanisme yang digunakan untuk memeriksa atau memproses request sebelum request tersebut diteruskan ke bagian aplikasi berikutnya. Jika ada request yang tidak memenuhi ketentuan middleware, request tersebut dapat diputus oleh middleware. Jika digambarkan secara sederhana, konsep middleware seperti gambar berikut:



Gambar 3.2: Konsep middleware

Karena middleware berada di antara request dan controller, middleware sangat berguna untuk menangani proses yang harus dilakukan sebelum endpoint dijalankan. Tanpa middleware, pemeriksaan yang sama harus dilakukan berulang kali pada setiap controller yang memerlukan perlindungan. Laravel memungkinkan middleware diterapkan dengan cakupan yang berbeda. Middleware dapat diterapkan secara global, sehingga dapat digunakan secara umum untuk request aplikasi. Middleware juga dapat diterapkan pada route tertentu, sehingga hanya endpoint yang membutuhkan pemeriksaan tersebut yang akan melewati pemeriksaan tersebut. Selain itu, beberapa middleware dapat dikelompokkan menjadi middleware group agar sekumpulan middleware dapat diterapkan secara bersamaan. Misalnya, sebuah aplikasi memiliki beberapa endpoint yang membutuhkan autentikasi:

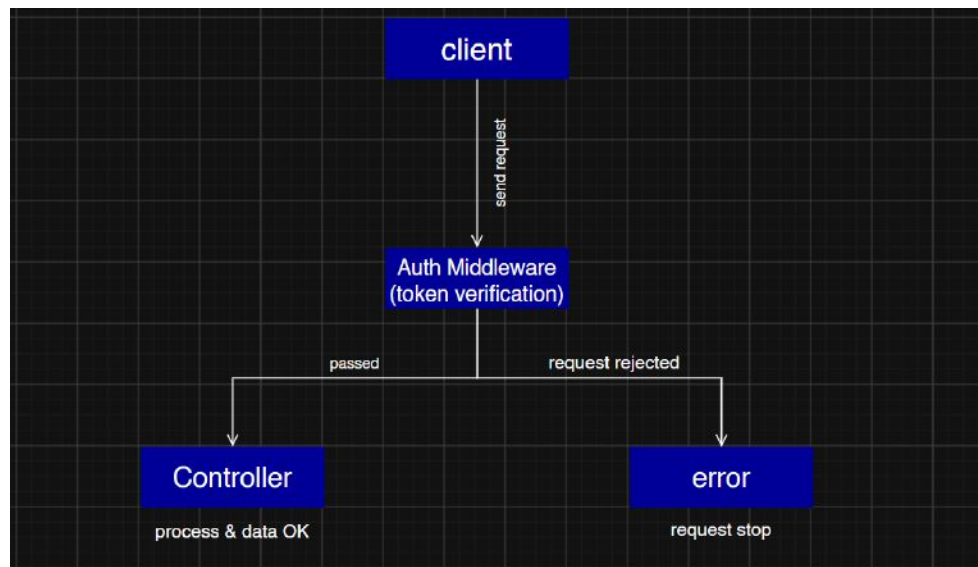
Protected API

```

|— GET    /api/me
|— POST   /api/logout
|— POST   /api/contents
|— PUT    /api/contents/{id}
|— DELETE /api/contents/{id}
  
```



Daripada melakukan autentikasi secara manual pada setiap controller, endpoint tersebut dapat dilindungi menggunakan **authentication middleware**. Middleware ini merupakan salah satu middleware penting yang digunakan untuk memastikan bahwa request berasal dari pengguna yang sudah terautentikasi. Pada implementasi JWT, client akan mengirimkan token JWT bersama request. Authentication middleware kemudian memeriksa token tersebut. Jika token valid, request dapat diteruskan ke controller. Jika token tidak tersedia atau tidak valid, request dapat dihentikan. Gambaran sederhananya seperti ini:



Gambar 3.3: Ilustrasi auth middleware

Dengan demikian, JWT tidak hanya digunakan saat login. JWT digunakan kembali ketika klien mengakses resource yang memerlukan autentikasi.

3.2. Public dan protected route.

Tidak semua endpoint dalam aplikasi memerlukan autentikasi. Beberapa endpoint memang harus bisa diakses oleh pengguna tanpa harus login. Endpoint seperti ini disebut sebagai public route. Sebaliknya, terdapat endpoint yang hanya boleh digunakan oleh pengguna yang telah melakukan autentikasi. Endpoint tersebut disebut protected route. Contohnya, pada aplikasi sistem perpustakaan yang kita bahas di modul ini, informasi mengenai katalog buku dan detail informasi pada buku umumnya boleh dilihat oleh siapa saja tanpa harus melakukan autentikasi, sehingga endpoint ini merupakan public route:

```

GET /api/books
GET /api/books/{book}
  
```



Sebaliknya, endpoint yang digunakan untuk meminjam dan mengembalikan buku sebaiknya dilindungi menggunakan authentication middleware, sehingga pengguna baru dapat meminjam atau mengembalikan buku setelah mereka login. Berikut endpointnya:

POST /api/members/{id}/borrow-book

PATCH /api/members/{id}/return-book/{book_id}

Pembagian tersebut tentu hanya sebagai contoh sederhana. Tidak semua aplikasi harus memiliki pembagian endpoint yang sama. Sebagai developer, Anda harus bisa menentukan endpoint mana yang membutuhkan autentikasi berdasarkan kebutuhan dan aturan akses pada aplikasi yang sedang dikembangkan. Contoh sederhana menambahkan authentication middleware pada route:

```
// authentication middleware to protect specific routes
Route::post('members/{id}/borrow-book', [MemberController::class, 'borrowBook'])
    ->middleware('auth:api');
Route::patch('members/{id}/return-book/{book_id}', [MemberController::class,
    'returnBook'])
    ->middleware('auth:api');

// example group middleware (authentication middleware)
Route::group(['middleware' => 'api', 'prefix' => 'auth'], function ($router) {
    Route::post('register', [AuthController::class, 'register']);
    Route::post('login', [AuthController::class, 'login']);
    Route::post('logout', [AuthController::class, 'logout'])->middleware('auth:api');
    Route::get('me', [AuthController::class, 'me'])->middleware('auth:api');
});
```

Gambar 3.4: Route API dengan middleware

Setiap endpoint route yang dipasang authentication middleware menandakan bahwa route tersebut merupakan protected route. Artinya, jika ada request yang masuk melalui route tersebut, middleware akan memeriksa terlebih dahulu apakah pengguna sudah terautentikasi. Jika valid, request akan diteruskan; jika tidak valid, middleware akan langsung memutus request.

3.3. Bearer token dan pengujian API

Setelah membahas middleware dan cara melindungi route menggunakan middleware, Anda mungkin bertanya-tanya bagaimana client memberi tahu server bahwa pengguna sudah melakukan autentikasi. Pada implementasi JWT, client mengirimkan token JWT yang diperoleh saat login pada request berikutnya. Token JWT umumnya dikirimkan melalui Authorization header dengan format **Authorization: Bearer <JWT>**. Contohnya: **Authorization: Bearer eyJhbGciOiJIUzI1Ni...** Kata Bearer menunjukkan bahwa token tersebut digunakan sebagai kredensial untuk



mengakses sumber daya yang dilindungi. Untuk lebih jelas, kita akan melakukan simulasi dan pengujian menggunakan Postman.

Karena sebelumnya kita sudah membuat controller (AuthController), sekarang kita harus membuat endpoint-nya dan memasang middleware pada route tertentu. Buka file api.php di folder routes dan sesuaikan seperti ini:

```
<?php

use App\Http\Controllers\BookController;
use App\Http\Controllers\MemberController;
use App\Http\Controllers\AuthController;
use Illuminate\Support\Facades\Route;

Route::apiResource('books', BookController::class);
Route::apiResource('members', MemberController::class);

// authentication middleware to protect specific routes
Route::post('members/{id}/borrow-book', [MemberController::class, 'borrowBook'])
    ->middleware('auth:api');
Route::patch('members/{id}/return-book/{book_id}', [MemberController::class, 'returnBook'])
    ->middleware('auth:api');

// example group middleware (authentication middleware)
Route::group(['middleware' => 'api', 'prefix' => 'auth'], function ($router) {
    Route::post('register', [AuthController::class, 'register']);
    Route::post('login', [AuthController::class, 'login']);
    Route::post('logout', [AuthController::class, 'logout'])->middleware('auth:api');
    Route::get('me', [AuthController::class, 'me'])->middleware('auth:api');
});
```

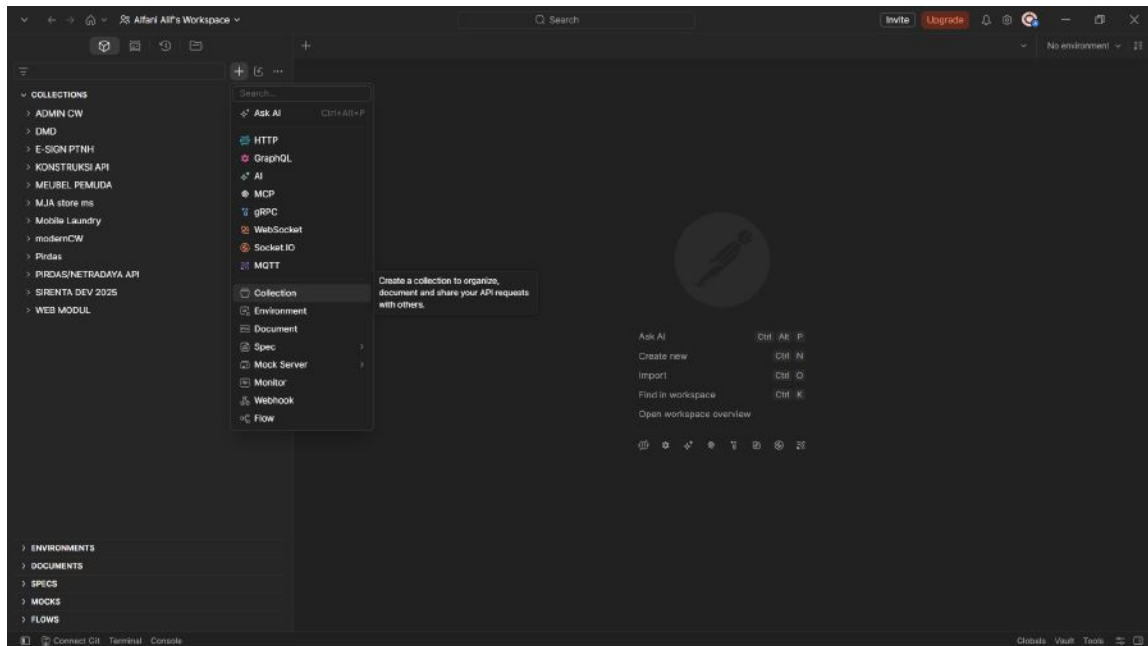
Gambar 3.5: Setup final route API dengan middleware

SEBELUM MELAKUKAN PENGUJIAN, PASTIKAN APLIKASI LARAVEL SUDAH DIJALANKAN. JIKA BELUM, SILAHKAN RUN DENGAN MENGETIKKAN COMMAND:

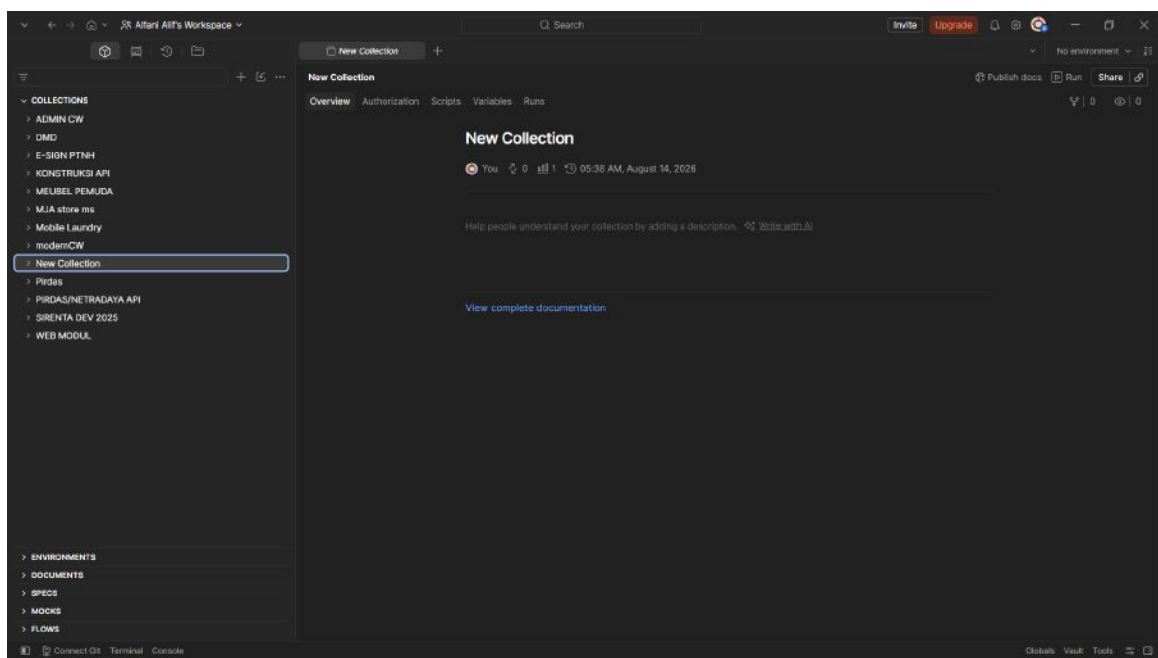
`php artisan serve`

Selanjutnya, silakan buka Postman. Jika di modul sebelumnya Anda sudah pernah melakukan pengujian, maka kali ini cukup meneruskannya saja. Agar lebih rapi dan terorganisir, buat satu collection di Postman.

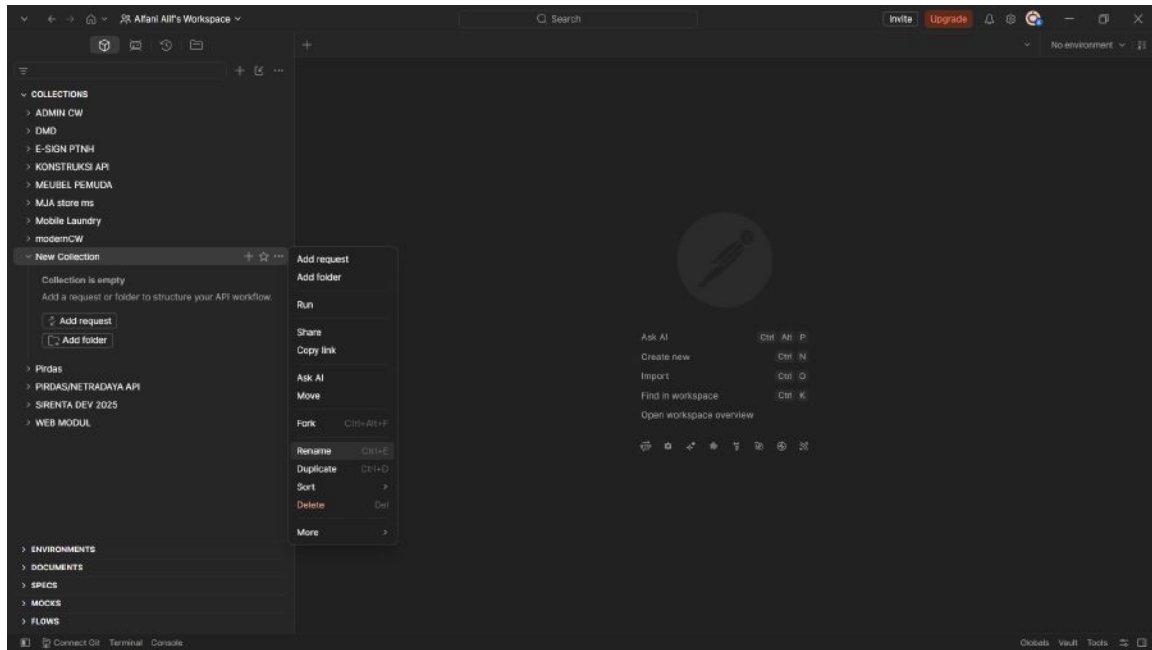




Jika sudah, maka akan muncul collection baru bernama "new collection".

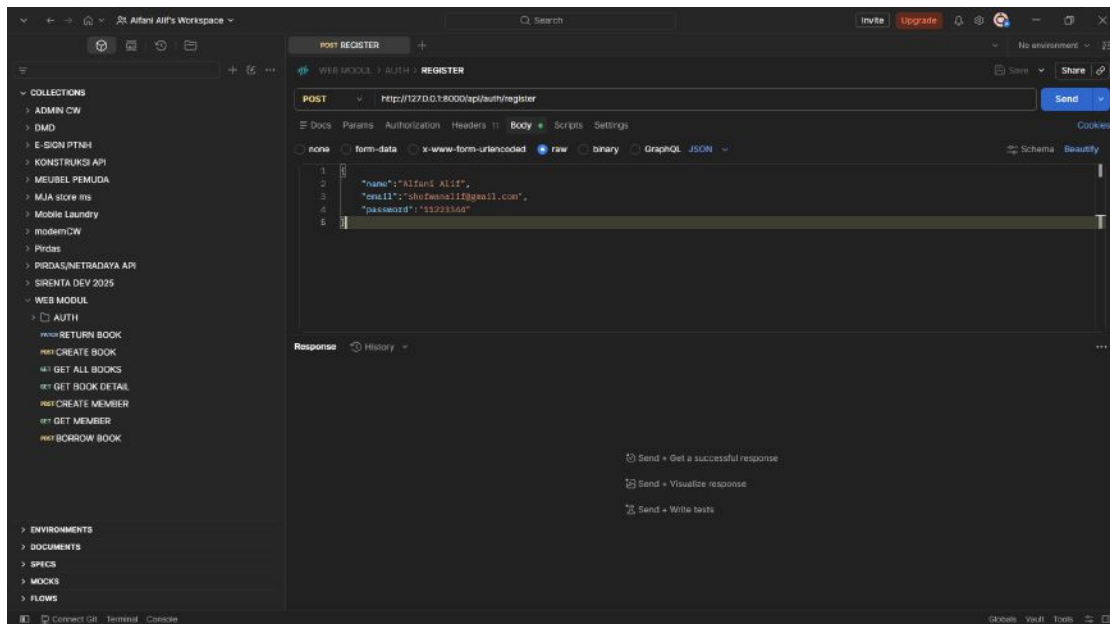


Selanjutnya rename dengan nama bebas, contoh disini di rename menjadi web modul. Nah di dalam collection inilah nantinya semua pengujian kita lakukan

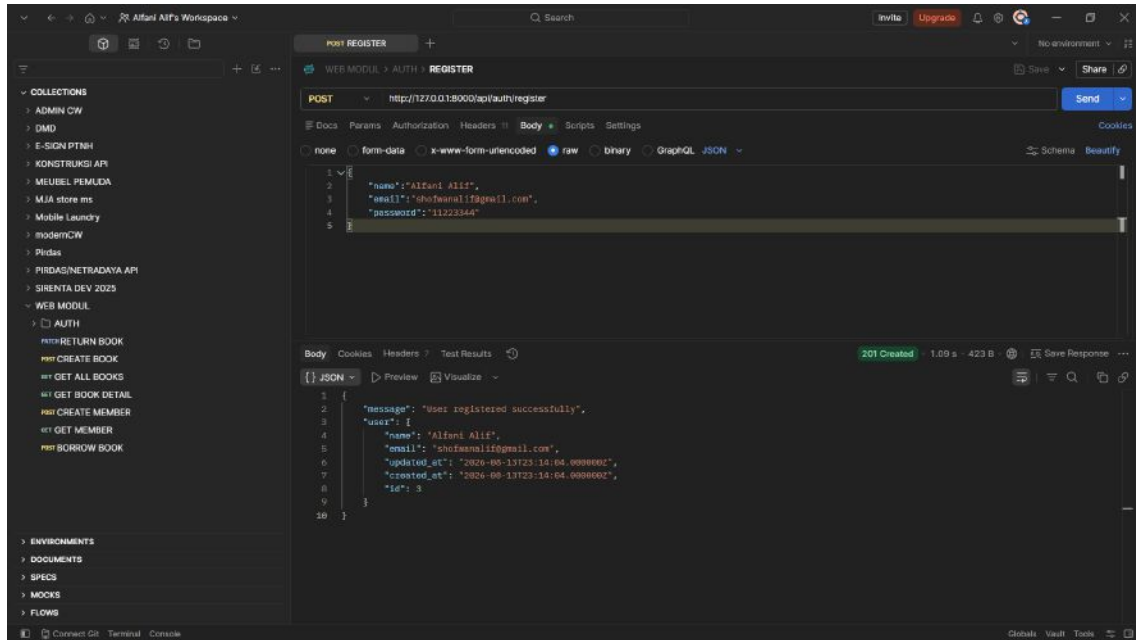


Jika pengujian di modul sebelumnya belum masuk ke collection, pindahkan pengujian tersebut ke collection dengan cara drag and drop dari luar collection ke dalam collection.

Di dalam collection, tambahkan request baru dengan method POST. Kali ini, kita akan menguji endpoint registrasi terlebih dahulu. Di tab body, pilih raw dan pilih tipe datanya menjadi JSON, lalu isi data yang diperlukan untuk registrasi. Jika semua data sudah terisi, klik "Send".



Jika berhasil, maka akan ada respons dengan status 201 yang artinya registrasi berhasil



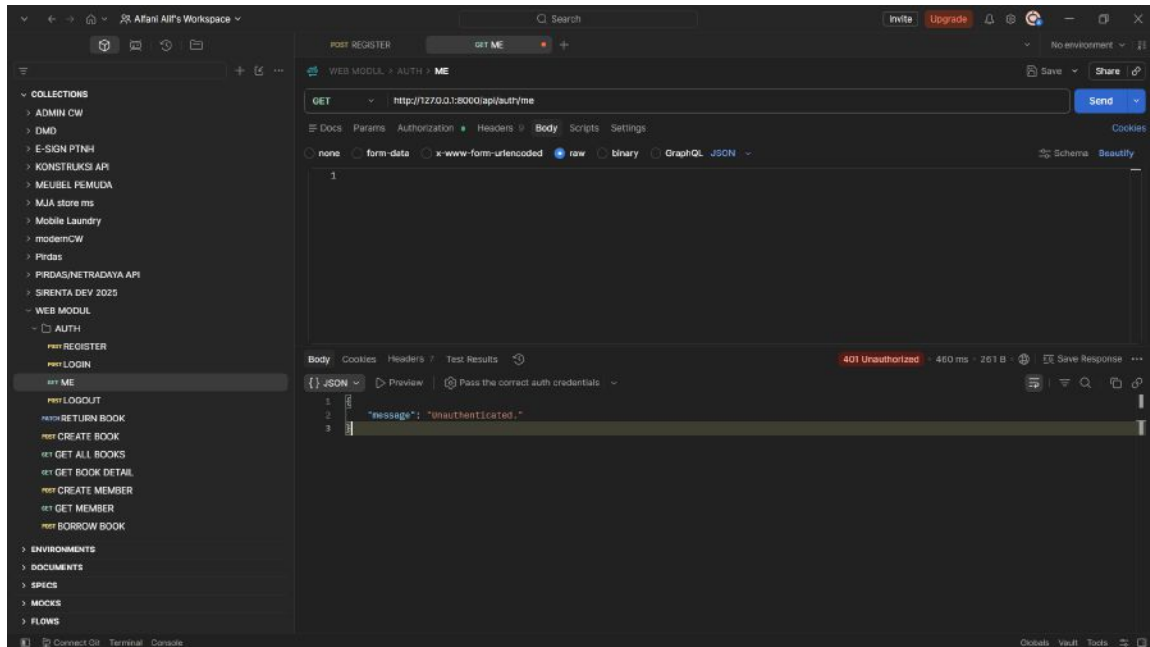
Pastikan juga di database user baru sudah terdaftar:

☐	Edit	Copy	Delete	3	Alifani Alif	shofwanalif@gmail.com	NULL	\$2y\$12\$SsQnieoFZ6G3kP2i4IAWHaeHLba85Y9UU3AMmnaJ2wh6...	NULL	2026-08-13 23:14:04	2026-08-13 23:14:04
--------------------------	------	------	--------	---	--------------	-----------------------	------	---	------	---------------------	---------------------

↑ ☐ Check all With selected: Edit Copy Delete Export



Selanjutnya kita akan menguji endpoint yang dilindungi untuk memastikan route-nya sudah benar-benar terlindungi. Contohnya di sini melakukan request ke **GET api/auth/me**



Hasilnya, request kita ditolak dengan respons "Unauthenticated". Hal ini menandakan bahwa route endpoint tersebut sudah berhasil dilindungi menggunakan authentication middleware, sehingga kita tidak bisa mengakses resources dari endpoint tersebut sebelum kita terautentikasi. Pastikan juga endpoint lain seperti endpoint untuk borrow book dan return juga tidak bisa diakses jika belum melakukan autentikasi.



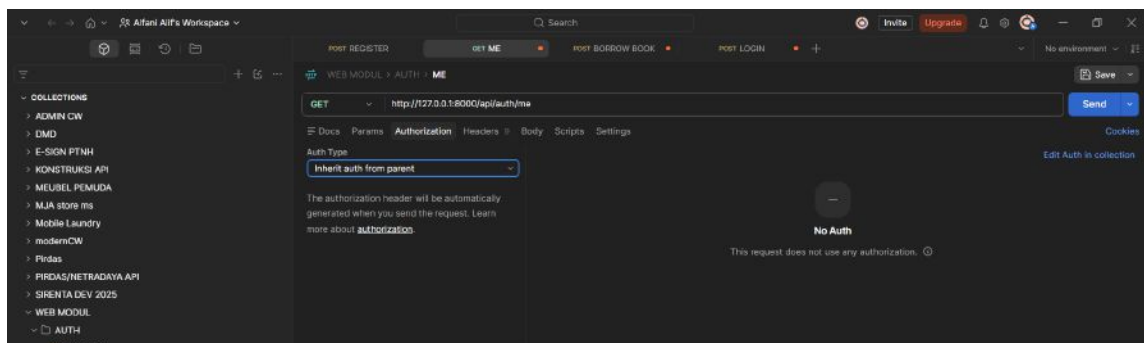


Sekarang request kita sudah berhasil mengakses resource yang sebelumnya tidak bisa kita akses. Step yang sama bisa kalian lakukan jika ingin mengirimkan request ke protected route yang lain

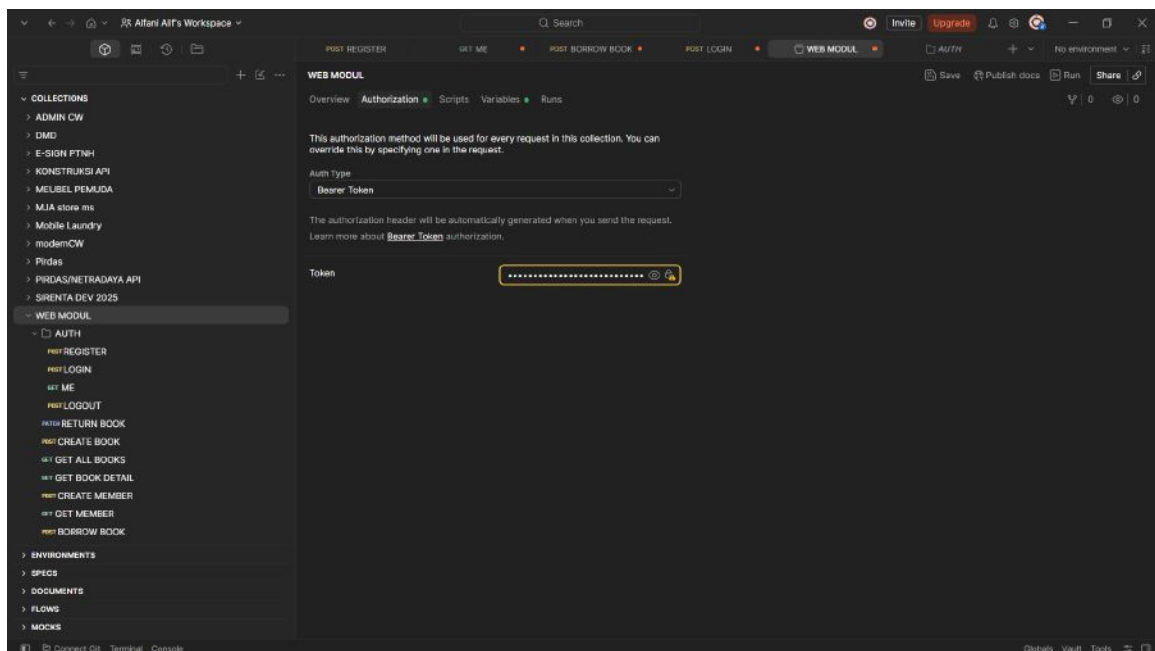
Sedikit tips Postman:

Daripada kita menyalin token setiap kali ingin mengirim request ke protected route, kita bisa memanfaatkan fitur Postman, yaitu *inherit auth from parent*. Dengan fitur ini, kita hanya mengubah bearer token yang ada di parent, yaitu collection kita. Caranya:

Ganti tipe endpoint yang memerlukan autentikasi menjadi inherit auth from parent



Nah sekarang, klik collection yang kita pu. Di sini contohnya collection bernama WEB MODUL. Klik tab authorization, ganti auth type-nya ke Bearer Token, lalu paste token-nya di sini. Secara otomatis, endpoint yang tadi memerlukan token akan mengambil token dari parent tanpa harus melakukan copy-paste manual berulang kali.



4. File storage dan file upload

4.1. Konsep file storage

File storage adalah mekanisme yang digunakan oleh aplikasi untuk menyimpan file pada media penyimpanan tertentu. Berbeda dengan data seperti nama, email, atau harga yang dapat disimpan langsung sebagai nilai pada database, file biasanya disimpan dalam sistem file, sedangkan database menyimpan informasi yang mengarah ke file tersebut. Sebagai contoh, sebuah buku di sistem perpustakaan memiliki gambar cover:

Tabel 4: Contoh tabel data Book dengan file storage

ID	TITLE	AUTHOR	FILE
1	Kungfu Panda	He Who Must Not Be Named	covers/kungfupanda.jpg

Data seperti id, title, dan author dapat disimpan dalam database. Sementara itu, file gambar cover disimpan di storage. Database tidak harus menyimpan isi binary dari gambar tersebut. Database cukup menyimpan path ke lokasi file. Laravel menyediakan abstraction bernama filesystem untuk mengelola penyimpanan file. Dengan abstraction tersebut, aplikasi tidak perlu menangani seluruh proses filesystem secara manual. Laravel juga menyediakan beberapa disk untuk menentukan lokasi penyimpanan file, yaitu local dan public. Disk local digunakan untuk menyimpan file pada lokasi yang bersifat internal bagi aplikasi. File yang disimpan di sana tidak dapat diakses secara otomatis melalui URL publik. Sementara itu, disk public digunakan ketika file memang perlu diakses oleh pengguna melalui web. Perbedaan tersebut penting karena tidak semua file dalam aplikasi seharusnya dapat diakses secara publik. Pemilihan storage juga harus disesuaikan dengan kebutuhan file. Pendekatan ini akan menjadi dasar ketika aplikasi mulai menerima file dari pengguna melalui proses unggah.

4.2. Public storage dan akses file

Setelah memahami konsep storage, kita tahu bahwa database hanya menyimpan path yang mengarah ke lokasi file. Jika menggunakan disk public, file yang disimpan secara default berada di direktori `storage/app/public`. Direktori ini bukan direktori publik yang dapat diakses melalui URL. Pertanyaannya adalah bagaimana file yang disimpan dapat diakses langsung oleh pengguna? Laravel menyediakan mekanisme symbolic link atau symlink. Melalui mekanisme ini, Laravel dapat menghubungkan satu lokasi filesystem dengan lokasi filesystem lainnya. Untuk mengimplementasikannya, Laravel menyediakan command `php artisan storage:link`. Command tersebut akan membuat hubungan antara `public/storage` → `storage/app/public`. Artinya, file yang berada di `storage/app/public/covers/kungfupanda.jpg` kini bisa diakses melalui `public/storage/covers/kungfupanda.jpg`. Dengan demikian, `storage:link` bukan

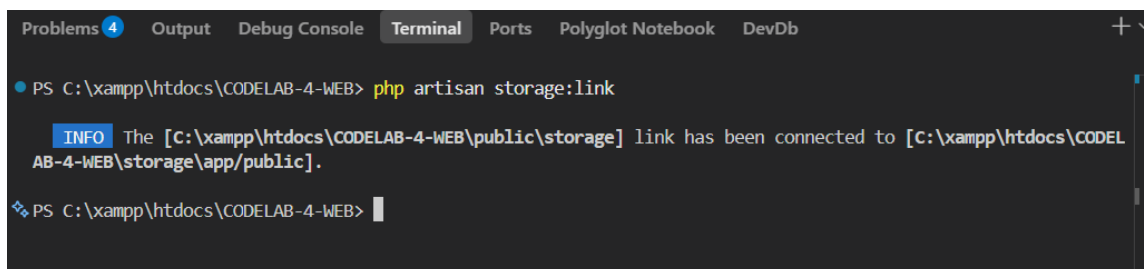


berarti memindahkan file dari storage ke public. File tetap berada di storage, sedangkan symbolic link membuat file tersebut dapat diakses melalui direktori public. Hal ini juga menunjukkan mengapa pemilihan disk perlu diperhatikan. Jika sebuah file bersifat public, seperti cover atau thumbnail konten, penggunaan public storage dapat sesuai dengan kebutuhan aplikasi. Namun, file yang bersifat sensitif atau tidak boleh diakses secara bebas sebaiknya tidak ditempatkan pada storage public tanpa mekanisme perlindungan tambahan.

4.3. File upload

Setelah memahami bagaimana file disimpan dan diakses, tahap berikutnya adalah mengimplementasikannya dengan membuat fitur file upload. File upload adalah proses ketika client mengirimkan file ke server melalui HTTP request. Server kemudian menerima file tersebut, melakukan validasi, menyimpannya di storage, dan apabila diperlukan, menyimpan path file ke dalam database. Berikut langkah - langkahnya:

Jalankan command `php artisan storage:link`



```

PS C:\xampp\htdocs\CODELAB-4-WEB> php artisan storage:link

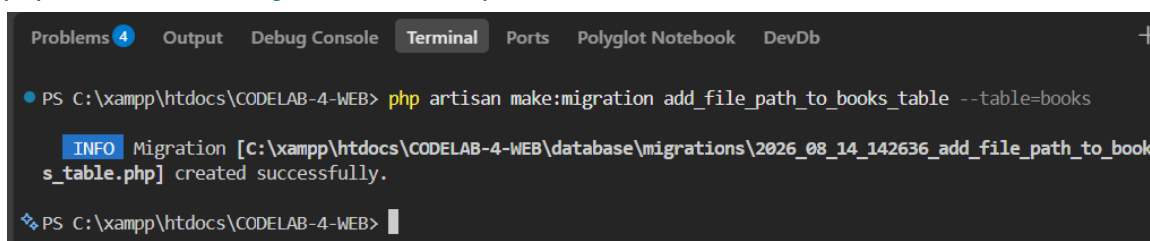
INFO The [C:\xampp\htdocs\CODELAB-4-WEB\public\storage] link has been connected to [C:\xampp\htdocs\CODELAB-4-WEB\storage\app/public].

PS C:\xampp\htdocs\CODELAB-4-WEB>
  
```

Gambar 4.1: Inisialisasi storage dengan symlink

Buat migrasi baru untuk menambahkan kolom file_path ke tabel books di database dengan menjalankan command

`php artisan make:migration add_file_path_to_books_table --table=books`



```

PS C:\xampp\htdocs\CODELAB-4-WEB> php artisan make:migration add_file_path_to_books_table --table=books

INFO Migration [C:\xampp\htdocs\CODELAB-4-WEB\database\migrations\2026_08_14_142636_add_file_path_to_books_table.php] created successfully.

PS C:\xampp\htdocs\CODELAB-4-WEB>
  
```

Gambar 4.2: Migrasi menambahkan kolom file_path

Modifikasi file migration yang baru dibuat menjadi seperti ini. Migration ini berfungsi untuk menambahkan kolom baru, yaitu file_path, pada tabel books.



```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('books', function (Blueprint $table) {
            $table->string('file_path')->nullable()->after('member_id');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('books', function (Blueprint $table) {
            $table->dropColumn('file_path');
        });
    }
};
```

Gambar 4.3: Kode final file migration final sebelum di-migrate

Setelah itu, jalankan command `php artisan migrate` untuk menerapkan perubahan ke database dan pastikan kolom baru bernama `file_path` sudah muncul pada tabel `books`.



Modifikasi model Book di app/Models/Book.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Book extends Model
{
    protected $fillable = ['title', 'author', 'is_available', 'member_id', 'file_path'];

    protected $casts = [
        'is_available' => 'boolean',
    ];

    public function member()
    {
        return $this->belongsTo(Member::class);
    }
}
```

Gambar 4.4: Kode final model Book sebelum di-migrate

Karena sebelumnya kita membuat request validation terpisah untuk method store di BookController, kita harus mengubah StoreBookRequest. Kode lengkap bisa diakses pada link berikut

<https://pastebin.com/EEAWvBT5>

Selanjutnya, kita juga harus memodifikasi file BookResource agar mengembalikan URL dan path file dalam responsnya. Kode lengkap bisa diakses pada link berikut

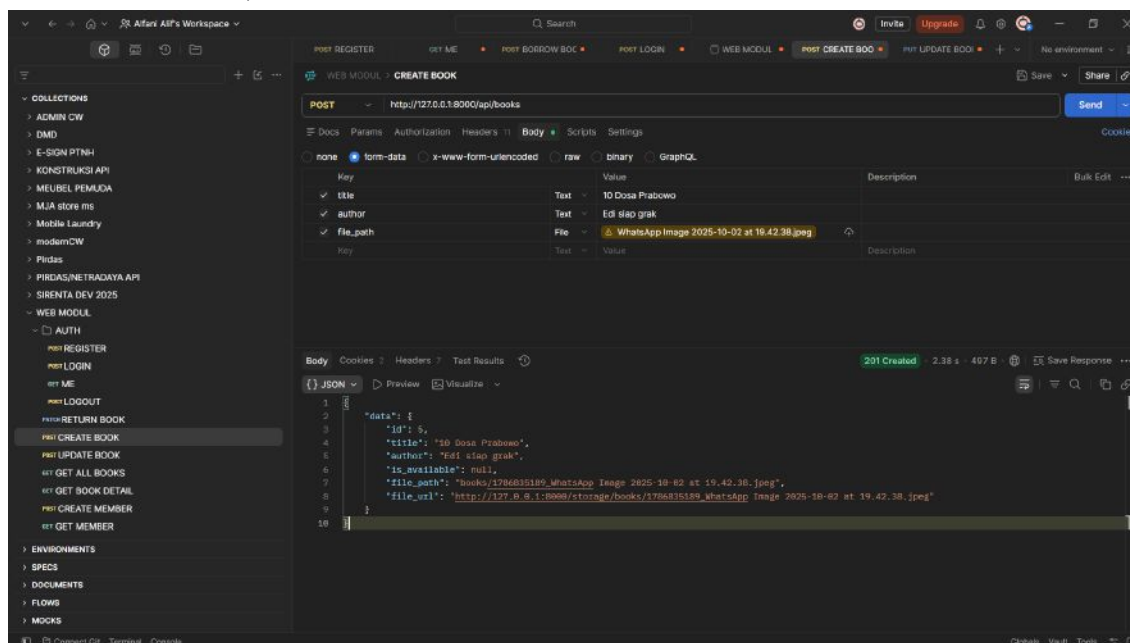
<https://pastebin.com/bctGwyga>

Terakhir barulah kita memodifikasi file BookController. Kode lengkap bisa diakses pada link berikut

<https://pastebin.com/CtLstGM0>



Jika sudah, langkah selanjutnya adalah melakukan pengujian dengan Postman untuk memastikan implementasi file upload kita sudah benar-benar berfungsi. Lakukan request ke endpoint POST /api/books (route endpoint-nya masih sama seperti pada modul sebelumnya).

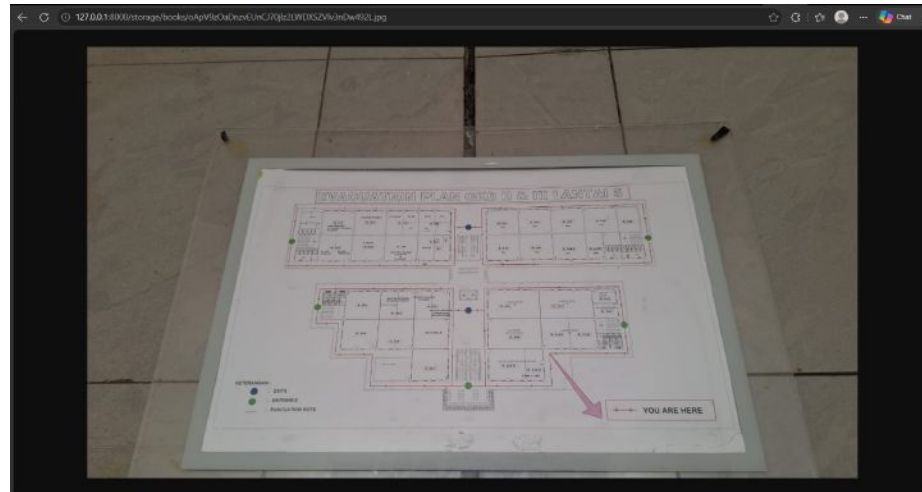
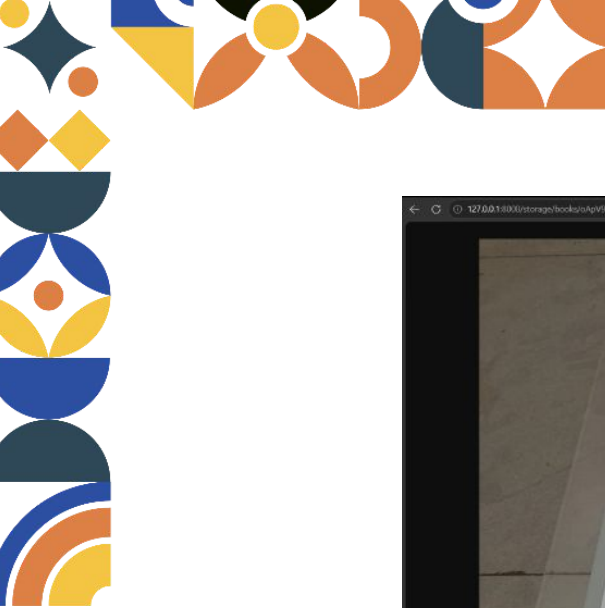


Gambar 4.5: Pengujian file upload menggunakan Postman

Di tab body, pilih form data dan isi key-nya sesuai dengan request yang sudah ditentukan dalam file `StoreBookRequ`. Type value-nya kalian bisa sesuaikan apabila strintype-nya bisa diisi dengan text, sedangkan `file_path` adalah bertipe file yang berarti kalian bisa mengupload sebuah file di situ. Klik Send. Jika berhasil, server akan merespons dan juga mengembalikan file path serta file url yang bisa kalian akses langsung dari browser.

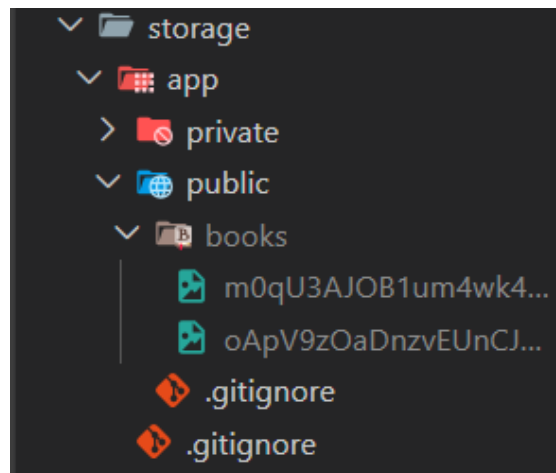
Contohnya, ketika file URL diakses langsung dari browser, akan menampilkan file yang sebelumnya kalian upload.





Gambar 4.6: Konfirmasi file dapat diakses dari browser

Selanjutnya kalian juga harus memastikan di local path kalian file-nya sudah tersimpan



Gambar 4.7: Konfirmasi file tersimpan di local path

Implementasikan juga pada method PATCH untuk memastikan datanya tetap bisa diupdate

5. CORS dan pengenalan integrasi API ke frontend

5.1. Cross-Origin Resource Sharing (CORS)

CORS (Cross-Origin Resource Sharing) adalah mekanisme keamanan pada browser yang mengatur apakah sebuah web page diperbolehkan melakukan request ke resource yang berada pada origin yang berbeda. Untuk memahami CORS, terlebih dahulu



perlu memahami konsep origin. Sebuah origin ditentukan oleh tiga komponen, yaitu scheme, host dan port. Sebagai contoh, aplikasi Laravel berjalan di <http://localhost:8000>.

Untuk detail komponennya:

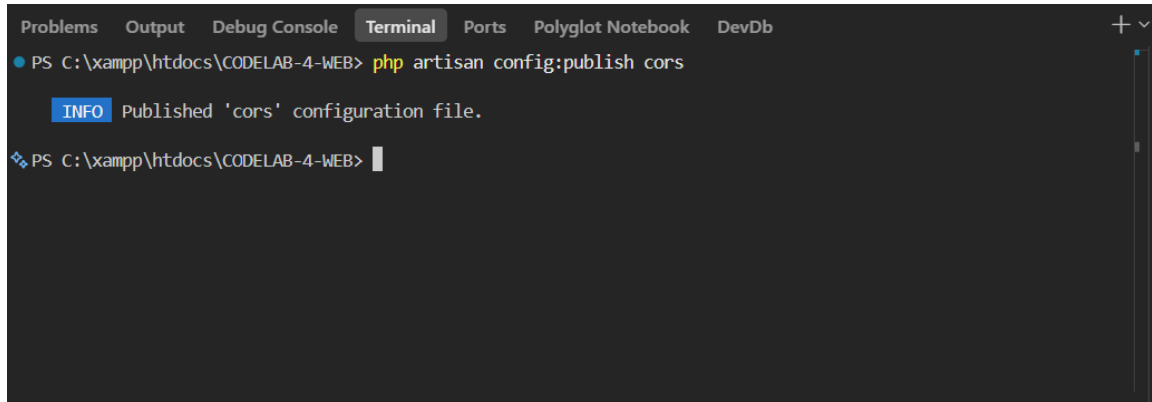
scheme	http
host	localhost
port	8000

Jika kedua aplikasi (frontend & backend) berjalan pada port yang sama, misalnya saat menggunakan Blade di Laravel, maka aplikasi dianggap memiliki origin yang sama sehingga browser tidak menganggap request tersebut sebagai cross-origin. Kondisinya akan berbeda jika kita menggunakan library frontend seperti React, Vue atau sejenisnya yang biasanya berjalan di <http://localhost:5173>, sedangkan Laravel sebagai backend API berjalan di <http://localhost:8000>. Dengan kondisi seperti itu, browser perlu memeriksa apakah backend mengizinkan origin frontend tersebut untuk melakukan permintaan. Jika origin tersebut diizinkan oleh backend, browser dapat memberikan respons kepada frontend. Jika tidak diizinkan, browser akan memblokir akses frontend terhadap respons tersebut.

Pada saat menggunakan Blade yang dirender langsung oleh Laravel, kebutuhan konfigurasi CORS umumnya tidak diperlukan karena keduanya dapat berjalan dari origin yang sama. Tetapi Laravel tetap menyediakan konfigurasi CORS untuk mengatur origin yang diperbolehkan mengakses resource tertentu jika dibutuhkan nanti. Konfigurasi tersebut tentunya harus disesuaikan dengan kebutuhan aplikasi. Contoh sederhana konfigurasi CORS di Laravel.

Jalankan command `php artisan config:publish cors`. Command tersebut akan otomatis generate file baru yaitu cors.php berlokasi di config/cors.php





```

Problems Output Debug Console Terminal Ports Polyglot Notebook DevDb
PS C:\xampp\htdocs\CODELAB-4-WEB> php artisan config:publish cors

INFO Published 'cors' configuration file.

PS C:\xampp\htdocs\CODELAB-4-WEB>

```

Gambar 5.1: Generate file config CORS

Buka file cors.php dan perhatikan pada bagian berikut



```

'paths' => ['api/*', 'sanctum/csrf-cookie'],

'allowed_methods' => ['http://localhost:3000'],

'allowed_origins' => ['*'],

'allowed_origins_patterns' => [],

'allowed_headers' => ['*'],

'exposed_headers' => [],

'max_age' => 0,

'supports_credentials' => true,

```

Gambar 5.2: Isi dan setting default file cors.php

Di bagian `allowed_origins`, Anda bisa mengizinkan origin mana saja yang boleh mengakses resource. Opsi `supports_credentials` digunakan untuk mengizinkan browser mengirimkan data kredensial otomatis (seperti Cookie, HTTP Authentication headers, atau sertifikat TLS klien) bersamaan dengan request API ke origin backend Laravel.

5.2. Pengenalan Integrasi API dengan client

Setelah memahami CORS, langkah berikutnya adalah memahami bagaimana API Laravel digunakan oleh client. REST API pada dasarnya tidak bergantung pada teknologi frontend tertentu. API hanya menerima request dan memberikan response sesuai



dengan endpoint yang tersedia. Artinya, API yang telah dibuat tidak hanya digunakan untuk satu jenis frontend.

Blade Laravel

Di Laravel, Blade merupakan template engine yang digunakan untuk menghasilkan tampilan HTML yang diproses di sisi server. Karena itu, jika menggunakan Blade, kita tidak perlu melakukan konfigurasi CORS karena berjalan pada origin yang sama. Contoh sederhana menggunakan Blade untuk menampilkan data buku:

Buat controller khusus untuk Blade dengan command `php artisan make:controller BookWebController`

```
PS C:\xampp\htdocs\CODELAB-4-WEB> php artisan make:controller BookWebController
INFO Controller [C:\xampp\htdocs\CODELAB-4-WEB\app\Http\Controllers\BookWebController.php] created successfully.
PS C:\xampp\htdocs\CODELAB-4-WEB>
```

Gambar 5.3: Generate file BookWebController

Di file BookWebController buat kodenya kurang lebih seperti ini



```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class BookWebController extends Controller
{
    public function index(Request $request)
    {
        // 1. Make an internal request to API endpoint
        $apiRequest = Request::create('/api/books', 'GET', [
            'page' => $request->get('page', 1)
        ]);

        // 2. Make an internal request using 'app()->handle()'.
        $apiResponse = app()->handle($apiRequest);

        // 3. Retrieve the JSON data from the API Controller response.
        $data = json_decode($apiResponse->getContent(), true);

        // 4. Extract the data
        $books = $data['data'] ?? [];
        $meta = $data['meta'] ?? null;
        $links = $data['links'] ?? null;

        return view('books.index', compact('books', 'meta', 'links'));
    }
}
```

Gambar 5.4: Kode file BookWebController

Buat route di routes/web.php



```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\BookWebController;

Route::get('/', function () {
    return view('welcome');
});

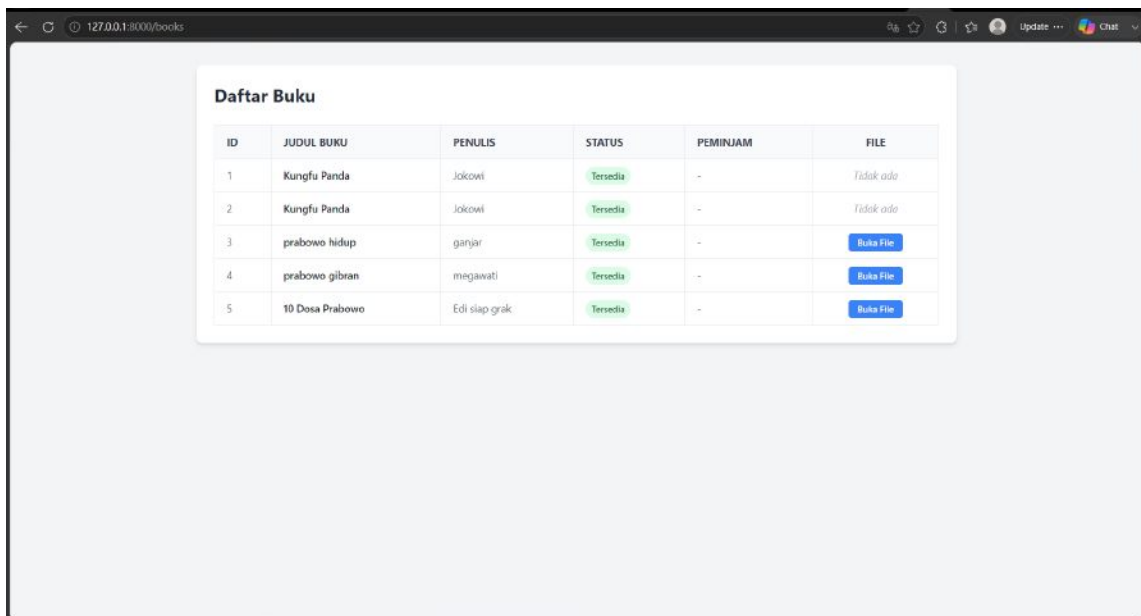
// Routes the browser URL (/books) to the BookWebController index method
Route::get('/books', [BookWebController::class, 'index'])->name('books.index');
```

Gambar 5.5: Membuat route khusus untuk Blade

Lalu buat direktori books di resources/views lalu buat file index.blade.php, kode lengkapnya ada di link berikut:

<https://pastebin.com/rMrX82pS>

Jika berhasil, maka ketika membuka url 127.0.0.1:8000/books atau localhost:8000/books, maka akan tampil seperti ini:



ID	JUDUL BUKU	PENULIS	STATUS	PEMINJAM	FILE
1	Kungfu Panda	Jokowi	Tersedia	-	Tidak ada
2	Kungfu Panda	Jokowi	Tersedia	-	Tidak ada
3	prabowo hidup	ganjar	Tersedia	-	Buka File
4	prabowo gibran	megawati	Tersedia	-	Buka File
5	10 Dosa Prabowo	Edi siap-grak	Tersedia	-	Buka File

Gambar 5.6: Konfirmasi halaman blade berhasil tampil beserta datanya

Catatan: pendekatan tersebut merupakan contoh sederhana saja; untuk versi yang lebih lengkap dan *best practice*, Anda bisa membaca dokumentasi resmi dari [Laravel](https://laravel.com).



5.3. Sekilas frontend framework

Selain Blade, API Laravel juga dapat digunakan oleh framework frontend seperti React, Vue, atau framework JavaScript lainnya. Perbedaan utamanya terletak pada cara frontend dan backend berkomunikasi. Pada pendekatan blade, Laravel bertanggung jawab untuk menghasilkan HTML yang akan ditampilkan. Sedangkan jika menggunakan framework frontend seperti React, Laravel tidak lagi bertanggung jawab untuk menghasilkan halaman. Laravel hanya bertindak sebagai backend API, sedangkan React bertanggung jawab mengelola antarmuka pada sisi frontend. Di ekosistem React, kita tidak menggunakan skema controller bertingkat seperti BookWebController. Komponen React berjalan langsung di sisi pengguna (Client-Side) dan mengambil data dari Laravel menggunakan fungsi `fetch()` atau library `Axios`. Berikut adalah contoh komponen React sederhana untuk menampilkan data buku dari REST API Laravel:

```
import React, { useState, useEffect } from 'react';

function BookList() {
  // 1. State to store book data from the Laravel API.
  const [books, setBooks] = useState([]);
  const [loading, setLoading] = useState(true);

  // 2. useEffect runs automatically when the React component first loads.
  useEffect(() => {
    // Call API
    fetch('http://127.0.0.1:8000/api/books')
      .then(response => response.json())
      .then(result => {
        setBooks(result.data); // Store the 'data' array from the Book Resource in
        // the state.
        setLoading(false);
      })
      .catch(error => console.error('Error fetching API:', error));
  }, []);

  if (loading) return <p>Loading data buku...</p>;

  // 3. Dynamically render HTML based on the 'books' state.
  return (
    <div style={{ padding: '20px' }}>
      <h2>Daftar Buku (React Frontend)</h2>
      <ul>
        {books.map(book => (
          <li key={book.id}>
            <strong>{book.title}</strong> - {book.author}
            {book.is_available ? ' (Tersedia)' : ' (Dipinjam)'}
          </li>
        ))}
      </ul>
    </div>
  );
}

export default BookList;
```



Gambar 5.7: Contoh sederhana kode framework React JS

Jika Anda masih asing dengan React, Anda bisa membaca dokumentasi resmi [React](#) dan juga Anda bisa membaca dokumentasi [Laravel](#) yang juga menjelaskan bagaimana jika Anda ingin memadukan langsung project Laravel dengan frontend framework javascript lain seperti React, Svelte atau Vue

CODELAB

Lakukan modifikasi pada codelab di modul sebelumnya untuk mengimplementasikan middleware dan file storage dengan mengikuti setiap step yang ada di modul ini, yaitu [implementasi autentikasi JWT menggunakan Laravel](#), middleware untuk melindungi endpoint tertentu pada bagian [public dan protected route](#), serta fitur file upload pada bagian [file upload](#).

Catatan: setelah implementasi selesai, lakukan pengujian menggunakan Postman seperti pada bagian [Bearer token dan pengujian API](#). Setelah itu, tunjukkan langsung kepada asisten.



DEMO

Pada modul ini, API FlipPlay dikembangkan lebih lanjut dengan menambahkan 4 fitur utama: autentikasi JWT, otorisasi role, unggah file, dan penyimpanan aset. Sistem akan memverifikasi identitas pengguna menggunakan JWT serta membatasi akses fitur seperti Creator Studio hanya untuk role tertentu melalui middleware. Selain itu, sistem dapat menerima dan menyimpan unggahan file, seperti gambar, ke server dengan aman, yang nantinya dapat diakses melalui URL publik. Seluruh fitur API ini akan diuji menggunakan Postman.

Ketentuan pengerjaan:

- Wajib melanjutkan API FlipPlay yang telah dibuat pada demo Modul 4
- Wajib menggunakan JWT untuk autentikasi
- Sistem wajib memiliki endpoint register, login, logout, dan `/api/me`
- Wajib menerapkan middleware untuk authentication dan authorization
- Wajib memiliki public route dan protected route
- Wajib menerapkan minimal dua role pengguna contohnya user dan creator dengan pembatasan akses terhadap fitur tertentu
- Wajib memiliki fitur Creator Studio yang hanya dapat diakses oleh role yang sesuai dan menyediakan operasi CRUD untuk konten
- Wajib memiliki fitur file upload pada salah satu resource
- File yang diunggah wajib memiliki validasi terhadap tipe dan ukuran file
- File yang telah diunggah harus dapat diakses melalui URL publik
- Wajib melakukan pengujian authentication, protected endpoint, role-based access, Creator Studio, dan file upload menggunakan Postman dengan Bearer Token
- menerapkan konfigurasi CORS sesuai kebutuhan aplikasi apabila client dan API berada pada origin yang berbeda



RUBRIK PENILAIAN

Aspek penilaian	Persentase (%)
Codelab	Total 10%
Implementasi lengkap sesuai petunjuk	5
Pengujian di Postman	5
Demo	Total 80%
Mengimplementasikan auth JWT	15
Memiliki 4 endpoint wajib	10
Mengimplementasikan autentikasi berbasis role	15
Mengimplementasikan middleware	10
Public route & protected route	5
Membuat fitur Creator Studio	15
Mengimplementasikan file upload	5
Melakukan pengujian di Postman	5
Pemahaman	Total 10%
Ketepatan menjawab pertanyaan	10

